

Additional FAANG problems

315. Count of Smaller Numbers After Self

To a Google L5/L6 engineer, this problem is a classic signal for **Inversion Counting**. While a junior might try to use a nested loop ($O(N^2)$), a senior engineer recognizes that we need a way to count elements in a dynamic range efficiently.

The most elegant and interview-stable way to solve this is by modifying the **Merge Sort** algorithm.

1. Problem Explanation

The Goal: Given an array `nums`, return a new array `counts` where `counts[i]` is the number of smaller elements to the **right** of `nums[i]`.

The Challenge: If you have `[5, 2, 6, 1]`: * For 5, look right: `{2, 6, 1}`. Two are smaller (2 and 1). Count = 2. * For 2, look right: `{6, 1}`. One is smaller (1). Count = 1. * For 6, look right: `{1}`. One is smaller (1). Count = 1. * For 1, look right: `{}`. Zero are smaller. Count = 0. Result: `[2, 1, 1, 0]`.

To do this in better than $O(N^2)$ time, we need to gather information about “smaller numbers” while we are already organizing the numbers.

2. Solution Explanation

The Intuition (The “Why”): Think about Merge Sort. In the “Merge” step, you have two sorted halves: **Left** and **Right**. `Left = [5, 8]`, `Right = [1, 6]`

When you pick a number from the **Left** half (say, 5) to put into the sorted array, and you notice that some numbers from the **Right** half (like 1) have *already* been placed into the sorted array, it means those numbers from the **Right** half were **smaller** than 5 and were originally to its **right**!

The Algorithm (The “How”): 1. **Tagging:** We can’t just move numbers around, or we’ll lose their original indices. We store them as pairs: `(value, original_index)`. 2. **Divide:** Split the array into halves recursively until you have single elements. 3. **Conquer (The Magic):** During the merge: * Maintain a counter `right_counter` which tracks how many elements from the **Right half** have been merged so far. * When an element from the **Left half** is merged, add the current `right_counter` to its total in the `counts` array.

Walkthrough with ASCII Visualization: Input: `nums = [5, 2, 6, 1]`

```

[ INITIAL DATA ]
Pairs: [(5,0), (2,1), (6,2), (1,3)]
Counts: [0, 0, 0, 0]

-----

[ STEP 1: MERGING (5,0) and (2,1) ]
Left: [(5,0)] Right: [(2,1)]

- Comparison: Is 5 <= 2? No.
- Take from Right: [(2,1)]
- right_counter = 1 (We just took one smaller element from the right!)

- Take from Left: [(5,0)]
- Action: counts[original_index 0] += right_counter (which is 1)
- Sorted result: [(2,1), (5,0)]
- Current counts: [1, 0, 0, 0]

-----

[ STEP 2: MERGING (6,2) and (1,3) ]
Left: [(6,2)] Right: [(1,3)]

- Comparison: Is 6 <= 1? No.
- Take from Right: [(1,3)]
- right_counter = 1

- Take from Left: [(6,2)]
- Action: counts[original_index 2] += 1
- Sorted: [(1,3), (6,2)]
- Current counts: [1, 0, 1, 0]

-----

[ STEP 3: FINAL MERGE ]
Left: [(2,1), (5,0)]
Right: [(1,3), (6,2)]
right_counter = 0

- Compare 2 and 1: 1 is smaller.
- Take 1 from Right. right_counter = 1.

- Compare 2 and 6: 2 is smaller.
- Take 2 from Left.
- Action: counts[original_index 1] += right_counter (1)
- Current counts: [1, 1, 1, 0]

- Compare 5 and 6: 5 is smaller.
- Take 5 from Left.

```

- Action: `counts[original_index 0] += right_counter (1)`
- Final counts: `[2, 1, 1, 0]`
- Take remaining 6 from Right.

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(N * \log N)$

-
- Merge Sort divides the array in half $\log N$ times.
 - At each level of the "tree," we perform a merge operation that takes $O(N)$ time.
 - Adding to the counts array is $O(1)$ during the merge.

[TC DERIVATION]

Level 1: `[ N ]` $\rightarrow N$ operations
 Level 2: `[ N/2 ] [ N/2 ]` $\rightarrow N$ operations
 Level 3: `[ N/4 ] [ N/4 ] [ N/4 ] ...` $\rightarrow N$ operations
 Total levels: $\log N$
 Total Complexity: $N * \log N$

SPACE COMPLEXITY: $O(N)$

-
- We store the 'counts' result array: $O(N)$.
 - We store the 'indexed_nums' (value, index) pairs: $O(N)$.
 - The recursion stack for merge sort goes $\log N$ deep.
 - Temporary arrays created during merging: $O(N)$.

[SC DERIVATION]

Result Array : `[] [] [] []` (N)
 Indexed Array : `[] [] [] []` (N)
 Temp Storage : `[] [] [] []` (N)
 Total: $O(N)$

4. Solution Code

Python Snippet

```
def countSmaller(nums):
    n = len(nums)
    counts = [0] * n
    # Store value with its original index to track it during sorting
```

```

indexed_nums = [(num, i) for i, num in enumerate(nums)]

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    sorted_arr = []
    l_ptr = r_ptr = 0
    # right_count tracks how many elements from the right side
    # were smaller than the current element on the left side
    right_count = 0

    while l_ptr < len(left) and r_ptr < len(right):
        if right[r_ptr][0] < left[l_ptr][0]:
            # Right element is smaller
            sorted_arr.append(right[r_ptr])
            right_count += 1
            r_ptr += 1
        else:
            # Left element is smaller or equal
            # We add the total count of smaller right elements
            # found SO FAR to this left element's original index
            counts[left[l_ptr][1]] += right_count
            sorted_arr.append(left[l_ptr])
            l_ptr += 1

    # Clean up remaining elements
    while l_ptr < len(left):
        counts[left[l_ptr][1]] += right_count
        sorted_arr.append(left[l_ptr])
        l_ptr += 1

    while r_ptr < len(right):
        sorted_arr.append(right[r_ptr])
        r_ptr += 1

    return sorted_arr

merge_sort(indexed_nums)
return counts

```

JavaScript Snippet

```
function countSmaller(nums) {
  const n = nums.length;
  const counts = new Array(n).fill(0);
  // Create an array of objects to keep track of original indices
  let indexedNums = nums.map((num, i) => ({ val: num, id: i }));

  function mergeSort(arr) {
    if (arr.length <= 1) return arr;

    const mid = Math.floor(arr.length / 2);
    const left = mergeSort(arr.slice(0, mid));
    const right = mergeSort(arr.slice(mid));

    return merge(left, right);
  }

  function merge(left, right) {
    const result = [];
    let l = 0, r = 0;
    let rightCount = 0;

    while (l < left.length && r < right.length) {
      if (right[r].val < left[l].val) {
        // Number on the right is smaller than number on left
        result.push(right[r]);
        rightCount++;
        r++;
      } else {
        // Number on left is processed, add the current rightCount
        counts[left[l].id] += rightCount;
        result.push(left[l]);
        l++;
      }
    }

    // Exhaust remaining left elements
    while (l < left.length) {
      counts[left[l].id] += rightCount;
      result.push(left[l++]);
    }

    // Exhaust remaining right elements
    while (r < right.length) {
      result.push(right[r++]);
    }
  }
}
```

```

    }

    return result;
}

mergeSort(indexedNums);
return counts;
}

```

Note 1: Terminology Addendum

Inversion Count: * **What it is:** A measure of how “unsorted” an array is. An inversion is a pair (i, j) where $i < j$ but $\text{nums}[i] > \text{nums}[j]$. * **Why it helps:** This problem is a specific version of counting inversions. Instead of a total count, we want a per-element count. * **How it applies:** Merge sort naturally compares elements from the “left” and “right,” making it the perfect vehicle for counting these inversions.

Fenwick Tree (Binary Indexed Tree) / Segment Tree: * **Alternative:** You could also solve this using a Fenwick Tree. You sort the unique numbers to “compress” them, then iterate through the array backwards. For each number, you query the tree for how many numbers smaller than it have already been added.

Note 2: Real-World Interview Variations

1. Google: “Reverse Pairs” (LeetCode 493) * **Prompt:** Count pairs where $i < j$ and $\text{nums}[i] > 2 * \text{nums}[j]$. * **The L5 Solve:** Use the exact same Merge Sort logic. The only difference is that during the merge, you run a separate small loop to count the $2 * \text{nums}[j]$ condition before actually merging the elements.

2. Bloomberg: “Global vs Local Inversions” (LeetCode 775) * **Prompt:** Is the number of global inversions equal to the number of local inversions? * **The L5 Solve:** While you could use Merge Sort to count global inversions, a senior engineer will notice that if a number is more than 1 index away from its sorted position, a non-local inversion **MUST** exist. This allows an $O(N)$ solution.

3. Meta: “Rank of Elements in Stream” * **Prompt:** You are receiving a stream of integers. At any point, return the rank of a number (how many numbers seen so far are smaller). * **The L5 Solve:** Since this is a stream, Merge Sort (which needs the full array) won’t work. You use a **Binary Search Tree (BST)** where each node stores the size of its left subtree, or a **Fenwick**

Tree. This demonstrates your ability to choose the right data structure for dynamic data.

299. Bulls and Cows

This problem is a test of **one-pass optimization** and **frequency tracking**. While many candidates solve this in two passes (one for Bulls, one for Cows), an L5/L6 engineer at Google will aim for a single-pass solution using a frequency array to demonstrate their ability to handle data streams efficiently.

1. Problem Explanation

The Premise: You are playing a game called “Bulls and Cows” (similar to Mastermind). You have a **secret** number and a **guess**. You need to provide a hint in the format “xAyB”.

- **Bulls (x):** Digits in the guess that are the **correct digit** and in the **correct position**.
- **Cows (y):** Digits in the guess that are the **correct digit** but in the **wrong position**.

The Constraint: Each digit in the secret can only be used once. Once a digit is counted as a Bull, it cannot be a Cow. If a digit appears multiple times in the guess but only once in the secret, only one Cow (or Bull) can be counted.

Example: Secret: 1807 Guess: 7810 * 8 is at index 1 in both: **1 Bull**. * 1, 0, and 7 exist in both but at wrong spots: **3 Cows**. Result: “1A3B”

2. Solution Explanation

To solve this like a Senior Engineer, we use a **Frequency Bucket** technique. We track the “balance” of digits we have seen so far.

The Strategy:

1. **Initialize:** A counter for **bulls**, a counter for **cows**, and an array **counts** of size 10 (for digits 0-9) initialized to 0.
2. **The One-Pass Walk:** Iterate through the strings. At each index *i*:
 - If `secret[i] == guess[i]`, increment **bulls**.
 - If they are different, we check the “debt” or “credit” of the digits:
 - If `counts[secret[i]] < 0`, it means this digit was previously seen in the **guess** but we needed it for the **secret**. Increment **cows**.

- If `counts[guess[i]] > 0`, it means this digit was previously seen in the `secret` but we needed it for the `guess`. Increment `cows`.
- Update the balance: Increment `counts[secret[i]]` (we have one available) and decrement `counts[guess[i]]` (we owe one).

Detailed ASCII Walkthrough Secret: 1123, Guess: 0111

Initial State:

`bulls = 0, cows = 0`

`counts = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]` (Indices 0-9)

STEP 1: Index 0

Secret: 1, Guess: 0

They are NOT equal.

- Is `counts[1] < 0`? No.

- Is `counts[0] > 0`? No.

Action:

`counts[1]` becomes 1 (Secret has 1 available)

`counts[0]` becomes -1 (Guess used a 0)

STEP 2: Index 1

Secret: 1, Guess: 1

They ARE equal.

Action:

`bulls = 1`

STEP 3: Index 2

Secret: 2, Guess: 1

They are NOT equal.

- Is `counts[2] < 0`? No.

- Is `counts[1] > 0`? YES! (From Step 1, secret provided a 1).

Action:

`cows = 1`

`counts[2]` becomes 1 (Secret has 2 available)

`counts[1]` becomes 0 (Guess used that available 1)

STEP 4: Index 3

Secret: 3, Guess: 1

They are NOT equal.

- Is `counts[3] < 0`? No.

- Is `counts[1] > 0`? No (it is now 0).

Action:
counts[3] becomes 1
counts[1] becomes -1 (Guess used another 1)

FINAL RESULT: "1A1B"

3. Time and Space Complexity Analysis

Time Complexity: $O(N)$ Where N is the length of the strings.

TC Visual Derivation:

String:	[	s1	s2	s3	...	sn	]
Loop:		1	2	3	...	n	

Each iteration involves:

- 1 Comparison: $O(1)$
- 2 Array Lookups: $O(1)$
- 2 Array Updates: $O(1)$

Total Work: $n * O(1) = O(N)$

Space Complexity: $O(1)$ Even though we use a `counts` array, its size is fixed (10 digits).

SC Visual Derivation:

Memory Allocated:

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9] (The 'counts' array)

Size of counts = 10 (Constant)

Size of bulls = 1 (Integer)

Size of cows = 1 (Integer)

The memory used does not grow, even if the
input strings are 1,000,000 characters long.
Result: $O(1)$

4. Solution Code

Python Implementation

```

def getHint(secret: str, guess: str) -> str:
    bulls = 0
    cows = 0
    # Array to track counts of digits 0-9
    # Positive: Secret has seen this digit and is looking for a guess match
    # Negative: Guess has seen this digit and is looking for a secret match
    counts = [0] * 10

    for s, g in zip(secret, guess):
        if s == g:
            bulls += 1
        else:
            # If counts[s] is negative, it means guess saw this digit earlier
            if counts[int(s)] < 0:
                cows += 1
            # If counts[g] is positive, it means secret saw this digit earlier
            if counts[int(g)] > 0:
                cows += 1

            # Update the balance
            counts[int(s)] += 1
            counts[int(g)] -= 1

    return str(bulls) + "A" + str(cows) + "B"

```

JavaScript Implementation

```

/**
 * Optimized One-Pass Solution
 */
var getHint = function(secret, guess) {
    let bulls = 0;
    let cows = 0;
    // Track balance of digits 0-9
    const counts = new Array(10).fill(0);

    for (let i = 0; i < secret.length; i++) {
        const s = parseInt(secret[i]);
        const g = parseInt(guess[i]);

        if (s === g) {
            bulls++;
        } else {
            // Check if secret digit was already requested by guess
            if (counts[s] < 0) cows++;
            // Check if guess digit was already provided by secret

```

```

        if (counts[g] > 0) cows++;

        // Increment for secret (contribution)
        counts[s]++;
        // Decrement for guess (requirement)
        counts[g]--;
    }
}

return bulls + "A" + cows + "B";
};

```

Note 1: Terms and Techniques

- **Frequency Bucket:** Using an array index to represent data values (like digits 0-9). This allows for $O(1)$ access instead of $O(\log N)$ for a hash map.
 - **One-Pass Algorithm:** A solution that processes the input data in a single traversal. This is highly preferred in stream processing where you might not be able to store the whole input.
 - **Balance Tracking:** Using positive/negative integers in a single array to track two different states (what the secret “gives” vs. what the guess “takes”).
-

Note 2: Real-World Interview Variations

1. Google: “Valid Wordle Guess” * The Setup: Similar to Bulls and Cows, but with 26 letters and colored tiles (Green, Yellow, Gray). * **L5 Solution:** Use the same one-pass approach. Bulls map to Green. Cows map to Yellow. You use a `counts` array of size 26 instead of 10.

2. Meta: “Finding Anagram Mappings” * The Setup: Given two strings that are anagrams, return the index mapping. * **L5 Solution:** This uses the frequency array logic to match characters. You would store lists of indices in your `counts` array (e.g., `counts['a'] = [0, 5, 12]`) to track which positions match.

3. Bloomberg: “Trade Matching Engine” * The Setup: You have “Buy” orders (Secret) and “Sell” orders (Guess). Match them if the price and volume are exactly the same (Bulls). If they are partially the same or for different stocks, categorize them (Cows). * **L5 Solution:** This involves the same “balance” logic. As trades come in, you increment/decrement a balance map. If a buy order comes in and the balance for that ticker is negative (sellers waiting), you have a match.

One relevant follow-up: how would you modify the one-pass approach if the “secret” and “guess” could contain any Unicode character instead of just digits 0-9?

Strobogrammatic Number II

An L5/L6 engineer looks at this problem and identifies it as a **Combinatorial Construction** problem. It isn't just about checking a property; it is about building a universe of valid strings. The senior approach is to recognize the **Symmetry** and the **Recursive Structure**—if you know all valid strobogrammatic numbers of length $n-2$, you can build all valid numbers of length n by wrapping them in pairs.

1. Problem Explanation

The Goal: A strobogrammatic number is a number that looks the same when rotated **180 degrees** (upside down). You are given an integer **n**, and you must return all strobogrammatic numbers that have a length of exactly **n**.

The “Digits” of Symmetry: Not all digits work. We only care about those that have a “rotational partner”: **0** rotates to **0** **1** rotates to **1** **8** rotates to **8** **6** rotates to **9** **9** rotates to **6**

Note: 2, 3, 4, 5, and 7 are invalid because they don't look like digits when flipped.

The “Length” Constraint: A number cannot start with **0** unless the number itself is just “0”. For $n > 1$, “010” is not a valid 3-digit number.

Visualizing Rotation:

```
Original:  6  0  9
Rotated:   6  0  9 (The 6 becomes a 9 at the end,
                   the 9 becomes a 6 at the start)
```

```
Original:  8 1 8
Rotated:   8 1 8 (All digits stay the same)
```

2. Solution Explanation

A senior engineer uses **Recursive Backtracking** or **Iterative Expansion** (Bottom-Up). Let's focus on the recursive intuition because it is the most elegant for this problem.

The Strategy: “Inside-Out” Construction

1. **Base Cases:**

- ### ASCII Diagram Walkthrough: $n = 4$ Level 1: Base Case ($n = 0$)

Level 2: Build $n = 2$ (Wrapping $n = 0$)

```
(1,1) -> "11"
(8,8) -> "88"
(6,9) -> "69"
(9,6) -> "96"
(0,0) -> "00" (Wait! We are at the outer layer for n=2,
                 but we will handle the 0-rule at the final step)
```

Level 3: Build $n = 4$ (Wrapping $n = 2$)

```
Wrap (1,1): "1111"
Wrap (8,8): "8118"
Wrap (6,9): "6119"
Wrap (9,6): "9116"
Wrap (0,0): "0110" (REJECTED: This is the outermost layer!)
```

Wrap (1,1): "1881" ... and so on.

```

+-----+
| TIME COMPLEXITY DERIVATION:  $O(5^{(n/2)})$  |
+-----+
| n = target length |
+-----+

```

1. The recursion depth is $n/2$ because we add 2 digits at a time.	
2. At each level, we branch into 5 possibilities:	
(0,0), (1,1), (8,8), (6,9), (9,6).	
3. The number of leaf nodes in our recursion tree is 5 to the	
power of $(n/2)$.	
4. Each string construction takes $O(n)$ time.	
TOTAL TIME: $O(n * 5^{(n/2)})$	
+-----+	
SPACE COMPLEXITY DERIVATION: $O(5^{(n/2)})$	
+-----+	
1. Result Array: Stores all valid numbers. There are roughly	
$4 * 5^{(n/2 - 1)}$ valid numbers.	
2. Recursion Stack: Depth of $n/2$.	
TOTAL SPACE: $O(n * 5^{(n/2)})$	
+-----+	

4. Solution Code

Python Snippet

```
def findStrobogrammatic(n):
    # helper function that takes 'm' (current length) and 'n' (target length)
    def generate(m, n):
        # Base Cases
        if m == 0: return [""]
        if m == 1: return ["0", "1", "8"]

        # Recursive call to get the "inner" numbers
        prev_nums = generate(m - 2, n)
        res = []

        for s in prev_nums:
            # Wrap each inner number with valid pairs
            if m != n:
                # We can only wrap with 0 if we aren't at the very outside
                res.append("0" + s + "0")

            res.append("1" + s + "1")
            res.append("8" + s + "8")
            res.append("6" + s + "9")
            res.append("9" + s + "6")
```

```

        return res

    return generate(n, n)
JavaScript Snippet
/**
 * @param {number} n
 * @return {string[]}
 */
var findStrobogrammatic = function(n) {
    const generate = (m, target) => {
        // Base case: length 0 is an empty string
        if (m === 0) return [""];
        // Base case: length 1 are the single symmetric digits
        if (m === 1) return ["0", "1", "8"];

        // Get strings of length m - 2
        let prev = generate(m - 2, target);
        let res = [];

        for (let s of prev) {
            // If we are not at the final outer layer, we can use '0'
            if (m !== target) {
                res.push("0" + s + "0");
            }
            // Always valid pairs
            res.push("1" + s + "1");
            res.push("8" + s + "8");
            res.push("6" + s + "9");
            res.push("9" + s + "6");
        }
        return res;
    };

    return generate(n, n);
};

```

Note 1: Terminology and Techniques

- **Recursive Construction:** Building a solution by solving a smaller version of the same problem. Here, length n is built directly from length $n-2$.
- **Backtracking:** Exploring all possible valid combinations. We “back-

track” once a branch is finished to try the next pair (e.g., after finishing the “1...1” branch, we start the “8...8” branch).

- **Permutations with Constraints:** This is a counting problem where the “Start with 0” rule is our primary constraint.

Note 2: Real-World Interview Variations

1. Google: “Upside-Down Password Generator” * Prompt: Create a tool that generates all 8-character passwords using numbers that look identical when the phone is flipped. The password cannot start with 0. *** Solution:** This is LC Strobogrammatic II with $n=8$. The interviewer is checking if you can handle the “Inside-Out” recursion logic.

2. Meta: “Mirror-Image IDs” * Prompt: You are designing a system for glass-printed IDs. Some IDs look the same from the front and back (assuming the digits themselves are symmetric). List all such 6-digit IDs. *** Solution:** Very similar, but “Mirror” symmetry is different from “Rotational” symmetry. For example, in a mirror, 6 does not become 9. You would simply change your “Pairs” map to only include digits like 1, 8, 0 and possibly others depending on the font. The recursive logic remains identical.

3. Bloomberg: “Rotational Arbitrage Strings” * Prompt: In a specialized trading terminal, certain codes must be “rotationally invariant” to avoid reading errors if the slip is held upside down. Given a length L , find all valid codes. *** Solution:** This is the exact problem. Bloomberg asks this to see if you can produce a clean, $O(N)$ space complexity (recursion stack) solution while managing a growing result set. You would solve it exactly as shown above.

691. Stickers to Spell Word

Solving “Stickers to Spell Word” (LeetCode 691) at an L5/L6 level at Google requires a shift from “finding a solution” to “optimizing an exponential search space.”

This is a Hard problem because it combines **Combinatorics** with **State Compression**. A senior engineer recognizes that the “state” isn’t the number of stickers we’ve used, but the **remaining characters** we still need to collect. We solve this using **Memoized Recursion (Top-Down Dynamic Programming)** on the string state.

1. Problem Explanation

The Core Question: You have several “stickers” (words). You can cut out individual letters from these stickers to spell a “target” word. You have an

infinite supply of each sticker. What is the minimum number of stickers needed to spell the target?

The Constraints: * You can't rearrange stickers to make new ones; you only extract letters. * If it's impossible, return -1. * Target length is small (up to 15), which is a massive hint that we can use **Bitmasking** or **String State Memoization**.

Visualizing the “Cutting” Process: Target: “basic” Stickers: [“ba”, “sic”, “abc”]

STRIKE 1: Use “ba”

Target “basic” -> Remaining “sic” (Stickers used: 1)

STRIKE 2: Use “sic”

Target “sic” -> Remaining “” (Stickers used: 2)

TOTAL: 2 stickers.

Why it's hard: There are many ways to start. Should I use a sticker that covers the ‘b’ or a sticker that covers the ‘a’? A greedy choice (picking the sticker with the most matching letters) might fail. We must explore all paths but skip the ones we've already calculated.

2. Solution Explanation

The L5 approach uses **Depth First Search (DFS) with Memoization**.

The Strategy: 1. **Pre-process Stickers:** We don't need the full sticker strings. We only care about the counts of letters that exist in the **target**. If a sticker has ‘z’ but the target doesn't, throw the ‘z’ away. 2. **Recursive State:** Our function `solve(remainder_of_target)` returns the min stickers to finish the job. 3. **Memoization:** Store `remainder_of_target` in a hash map so we don't re-calculate. 4. **Optimization (The First-Letter Rule):** To avoid redundant branches, we only consider stickers that contain the **very first letter** of our current target string. This drastically prunes the search tree.

ASCII Walkthrough: Target: “theapple” Stickers: [“the”, “ap”, “ple”]

DFS Level 0: “theapple”

```
|
|-- Try "the" (contains 't', the first letter)
|   Remaining: "apple"
|   |
|   |-- Try "ap" (contains 'a', the first letter)
|   |   Remaining: "ple"
|   |   |
|   |   |-- Try "ple" (contains 'p', the first letter)
```

```

|   |           Remaining: "" -> RETURN 1
|   |
|   |   RETURN 1 + 1 = 2
|
|   RETURN 1 + 2 = 3

```

Final Result: 3

Non-trivial Step: Character Subtraction When we apply sticker “ap” to “apple”, we don’t just remove the prefix. We remove ‘a’ and one ‘p’ from anywhere in the string. “apple” minus “ap” -> “ple”.

3. Time and Space Complexity Analysis

Let N be the number of stickers and T be the length of the target.

```

=====
TIME COMPLEXITY: O(N * 2^T * T)
=====

```

Derivation Diagram:

[State Space]

The number of possible substrings/subsequences of 'target' is 2 raised to the power of T.

Example: target "abc" has states "", "a", "b", "c", "ab", "ac", "bc", "abc".

Total states = 2^T

[Per State Work]

For each state, we iterate through N stickers.

For each sticker, we perform string manipulation of length T.

Total Time = (Total States) * (Stickers) * (String Work)

Total Time = $2^T * N * T$

```

=====
SPACE COMPLEXITY: O(2^T)
=====

```

Derivation Diagram:

[Memoization Map]

We store the result for every unique string state we encounter.

In the worst case, this is the same as the number of states.

Space = $O(2^T)$

[Recursion Stack]

The depth of the recursion is at most T (if each sticker provides only 1 useful letter).

Space = $O(T)$

Dominant Space = $O(2^T)$

=====

4. Solution Code

Python

```
from collections import Counter

def minStickers(stickers, target):
    # Step 1: Count characters for each sticker,
    # but only for characters present in the target.
    target_count = Counter(target)
    sticker_counts = []
    for s in stickers:
        s_count = Counter(s) & target_count
        if s_count:
            sticker_counts.append(s_count)

    # Memoization map
    memo = {"": 0}

    def solve(t_str):
        if t_str in memo:
            return memo[t_str]

        # Optimization: Only use stickers that contain the FIRST letter
        # of the current target string. This prunes the search tree.
        first_char = t_str[0]
        res = float('inf')

        for s_count in sticker_counts:
            if s_count[first_char] > 0:
                # Calculate the new remaining target string
                new_t = t_str
                for char, count in s_count.items():
                    # Remove 'count' occurrences of 'char' from new_t
                    new_t = new_t.replace(char, "", count)

                sub_res = solve(new_t)
```

```

        if sub_res != -1:
            res = min(res, 1 + sub_res)

    memo[t_str] = res if res != float('inf') else -1
    return memo[t_str]

# Sort target so memoization keys are consistent
return solve("".join(sorted(target)))

```

JavaScript

```

/**
 * L5/L6 Approach: Memoized DFS with Character Counting
 * Time:  $O(N * 2^T * T)$ , Space:  $O(2^T)$ 
 */
function minStickers(stickers, target) {
    const n = stickers.length;
    // Pre-process stickers into frequency maps
    const stickerCounts = stickers.map(s => {
        const count = {};
        for (const char of s) {
            count[char] = (count[char] || 0) + 1;
        }
        return count;
    });

    const memo = new Map();
    memo.set("", 0);

    function solve(t) {
        if (memo.has(t)) return memo.get(t);

        // Optimization: Find stickers that contain the first letter
        const firstChar = t[0];
        let minNeeded = Infinity;

        for (let i = 0; i < n; i++) {
            const sCount = stickerCounts[i];
            if (sCount[firstChar]) {
                // Generate the new target string after using this sticker
                let nextTarget = "";
                // Temporary copy of sticker count to track usage
                const remainingInSticker = { ...sCount };

                for (const char of t) {
                    if (remainingInSticker[char] > 0) {

```

```

        remainingInSticker[char]--;
    } else {
        nextTarget += char;
    }
}

const result = solve(nextTarget);
if (result !== -1) {
    minNeeded = Math.min(minNeeded, 1 + result);
}
}

const final = minNeeded === Infinity ? -1 : minNeeded;
memo.set(t, final);
return final;
}

// Sort target alphabetically to ensure "apple" and "pleap" are the same state
const sortedTarget = target.split('').sort().join('');
return solve(sortedTarget);
}

```

Note 1: Terms and Techniques

- **Memoization:** Storing the results of expensive function calls and returning the cached result when the same inputs occur again. Here, it prevents us from re-solving the same “remaining target string.”
 - **Pruning:** A technique in search algorithms that avoids exploring branches that cannot possibly lead to an optimal solution. The “First-Letter Rule” is a form of pruning.
 - **State Compression:** Representing a complex state (like a set of required letters) as a simple, comparable object (like a sorted string or a bitmask integer).
-

Note 2: Real-World & Interview Variations

1. **Google: “Minimum Taps to Water a Garden” (LeetCode 1326)** *
The Variation: Instead of letters, you have ranges (intervals) that cover a line. What’s the min number of ranges to cover the whole line? * **Solution:** While similar in “minimum count,” this is usually solved greedily using a variation of the **Jump Game** or via **Interval DP**.

2. Meta: “Change-making Problem” (Coins) * The Variation: Minimum coins to reach a target sum. * **Solution:** This is the 1D version of Stickers. Instead of character counts, you have integers. The “Stickers” problem is essentially a **Multidimensional Change-making Problem**.

3. Bloomberg: “Spellchecker / Word Suggestion” * The Variation: Given a scrambled or partial word, what are the minimum operations to match it to a dictionary? * **Solution:** This often uses **Edit Distance (Levenshtein)** logic. A senior engineer would solve this using a **Trie (Prefix Tree)** to search for valid dictionary words that can be formed from the available “stickers” (characters).

670. Maximum Swap

To a Google L5/L6 engineer, “Maximum Swap” is a **Greedy Optimization** problem. A senior engineer doesn’t just look for a swap; they look for the *most impactful* swap. The goal is to move the largest possible digit as far to the left as possible to maximize the “place value” of that digit.

1. Problem Explanation

The Goal: You are given a non-negative integer. You are allowed to swap **exactly two digits** at most once. Your objective is to find the maximum possible value you can create.

The “Why” behind the logic: To make a number larger, you want larger digits in higher place values (the left side). * If you have 2736, swapping 2 and 7 gives 7236. * Swapping 2 and 6 gives 6732. * 7236 is better because the 7 moved to the thousands place.

The Strategy: 1. We want to find a digit that has a larger digit appearing somewhere to its right. 2. We want that “larger digit” to be as big as possible (e.g., a 9 is better than an 8). 3. If there are multiple 9s to the right, we want the **rightmost** one, because swapping it leaves the other large digits in high place values.

2. Solution Explanation (Greedy Bucketing)

We use a “Last Occurrence” map to store the last index of every digit (0-9) present in the number.

The Algorithm Steps:

1. Convert the number into an array of digits.

2. Record the **last index** of every digit (0-9).
3. Scan the number from left to right.
4. For each digit, check if a larger digit (from 9 down to current+1) exists somewhere to its right.
5. The first time we find such a pair, we swap them and immediately return the result.

Step-by-Step Visualization Input: 2736

```
=====
INITIAL STATE
=====
Digits: [2, 7, 3, 6]
Indices: 0  1  2  3

Last Occurrence Map:
Digit 2: Index 0
Digit 7: Index 1
Digit 3: Index 2
Digit 6: Index 3

=====
SCANNING FROM LEFT
=====

Index 0: Digit is '2'
- Is there a '9' at an index > 0? No.
- Is there an '8' at an index > 0? No.
- Is there a '7' at an index > 0? YES! (Index 1)

ACTION: Swap Index 0 ('2') and Index 1 ('7').
Result: [7, 2, 3, 6] -> 7236.
STOP.
```

Input: 1993 (Handling the “Rightmost” rule)

```
=====
INITIAL STATE
=====
Digits: [1, 9, 9, 3]
Indices: 0  1  2  3

Last Occurrence Map:
Digit 1: Index 0
Digit 3: Index 3
Digit 9: Index 2 (The LAST index of 9 is 2, not 1)
```

SCANNING FROM LEFT

Index 0: Digit is '1'

- Is there a '9' at index > 0? YES! (Index 2)

ACTION: Swap Index 0 and Index 2.

Result: [9, 9, 1, 3] -> 9913.

(Why Index 2 and not Index 1? If we swapped with Index 1, we'd get 9193. 9913 is clearly better!)

3. Time and Space Complexity Analysis

Let N be the number of digits in the input.

TIME COMPLEXITY DERIVATION: $O(N)$

- | | |
|---------------------------------------|-----------------------------|
| 1. Convert number to list: | ---> $O(N)$ |
| 2. Fill Last Occurrence Map: | |
| Iterate N times to find last indices. | ---> $O(N)$ |
| 3. Scan digits: | |
| Iterate N times. | ---> $O(N)$ |
| Inside scan, loop 9 -> 0: | ---> $O(9)$ which is $O(1)$ |

Total Time = $O(N) + O(N) + O(N)$

Simplified = Order of N

SPACE COMPLEXITY DERIVATION: $O(N)$

What are we storing?

- | | |
|------------------------------|------------------------------|
| 1. List of digits: | ---> $O(N)$ |
| 2. Last Occurrence Map: | |
| Always size 10 (digits 0-9). | ---> $O(10)$ which is $O(1)$ |

Total Space = $O(N) + O(1)$

Simplified = Order of N

4. Solution Code

Python Implementation


```

def maximumSwap(num):
    # Convert number to list of digits for easy swapping
    digits = list(str(num))

    # Step 1: Map each digit to its last seen index
    # Example: 1993 -> { '1': 0, '9': 2, '3': 3 }
    last = {int(d): i for i, d in enumerate(digits)}

    # Step 2: Scan from left to right
    for i, d in enumerate(digits):
        current_digit = int(d)

        # Step 3: Look for a larger digit (9 down to current+1)
        # that appears after the current index
        for bigger_digit in range(9, current_digit, -1):
            if last.get(bigger_digit, -1) > i:
                # Perform the swap
                target_idx = last[bigger_digit]
                digits[i], digits[target_idx] = digits[target_idx], digits[i]

        # Convert back to integer and return immediately
        return int("".join(digits))

    # If no swap was made, return the original number
    return num

```

JavaScript Implementation

```

/**
 * @param {number} num
 * @return {number}
 */
var maximumSwap = function(num) {
    const digits = num.toString().split('');

    // Step 1: Create the 'last occurrence' bucket
    const last = new Array(10).fill(-1);
    for (let i = 0; i < digits.length; i++) {
        last[parseInt(digits[i])] = i;
    }

    // Step 2: Greedy check
    for (let i = 0; i < digits.length; i++) {
        const currentDigit = parseInt(digits[i]);

        // Step 3: Check if any digit from 9 down to current exists later

```

```

    for (let d = 9; d > currentDigit; d--) {
      if (last[d] > i) {
        // Swap digits
        let temp = digits[i];
        digits[i] = digits[last[d]];
        digits[last[d]] = temp;

        return parseInt(digits.join(''));
      }
    }

    return num;
  };

```

Note 1: Terminology & Techniques

- **Greedy Bucketing:** Instead of comparing every digit with every other digit ($O(N^2)$), we pre-calculate the “Best Case” (the last positions of 0-9) so that we can make a global decision in $O(1)$ at each step.
 - **Place Value Priority:** The fundamental concept that a change at the thousands place is 10x more valuable than a change at the hundreds place. This is why we scan from **left to right** and stop at the very first valid swap.
-

Note 2: Real-World Interview Variations

1. **Variation: Maximum Number after K Swaps (Google - Hard)** * **The Problem:** Same as this, but you can swap digits up to K times. * **How to solve:** Greedy doesn’t always work here because a swap now might prevent a better swap later. You must use **Backtracking (DFS)** to explore multiple swap paths, but prune the search by only swapping the current index with its maximum possible value to the right.
2. **Variation: Largest Number after Removing K Digits (Meta)** * **The Problem:** Given 1432219 and $K=3$, remove 3 digits to make the smallest/largest number. * **How to solve:** Use a **Monotonic Stack**. As you iterate, if the current digit is larger than the previous (for max) or smaller (for min), you pop the stack. This maintains a “sorted” feel to the digits.
3. **Variation: Reorganize String (Bloomberg)** * **The Problem:** Rearrange characters so that no two identical characters are adjacent. * **How to solve:** Use a **Max-Heap** based on character frequency. Always pick the most

frequent character first, then the second most frequent, to ensure you don't "run out" of different characters to separate the duplicates.

50. Pow(x, n)

To solve `Pow(x, n)` at a Google L5/L6 level, an engineer doesn't just write a loop. They immediately identify that a linear approach (multiplying `x` by itself `n` times) is unacceptable for large values of `n`. Instead, they reach for a technique called **Binary Exponentiation** (or "Exponentiation by Squaring").

1. Problem Explanation

The Goal: Calculate `x` raised to the power `n` (expressed as `x^n`).

The Constraint Trap: `n` can be a very large integer (up to 2,147,483,647) or a negative integer. * If `n` is 2 billion, a simple loop `for i in range(n): res *= x` will take several seconds and likely time out. * If `n` is negative, `x^-n` is equivalent to `1 / (x^n)`. * We must handle the edge case where `n` is the minimum possible 32-bit integer, as converting it to positive might cause an overflow in some languages.

The Intuition: Instead of doing `2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2` (8 operations), we can observe that: $2^8 = (2^4) * (2^4)$ $2^4 = (2^2) * (2^2)$ $2^2 = (2^1) * (2^1)$ By calculating the square at each step, we reach the answer in only 3 steps.

2. Solution Explanation

We use **Binary Exponentiation**. This approach follows a "Divide and Conquer" strategy.

The Core Logic For any `x^n`: 1. If `n` is **Even**: $x^n = (x * x)^{(n / 2)}$ 2. If `n` is **Odd**: $x^n = x * (x * x)^{((n - 1) / 2)}$

ASCII Visualization: Calculating 2^{10}

Target: 2^{10}

STEP 1: `n=10` (Even)

Logic: $2^{10} = (2*2)^5 = 4^5$

Current Base: 4, Current `n`: 5

STEP 2: `n=5` (Odd)

Logic: $4^5 = 4 * (4*4)^2 = 4 * 16^2$

Keep the '4' on the side.
Current Base: 16, Current n: 2

STEP 3: n=2 (Even)
Logic: $16^2 = (16*16)^1 = 256^1$
Current Base: 256, Current n: 1

STEP 4: n=1 (Odd)
Logic: $256^1 = 256 * (256*256)^0$
Anything to the power 0 is 1.

RESULT ASSEMBLY:
From Step 2, we had a '4' waiting.
From Step 4, we have '256'.
Final = $4 * 256 = 1024$.

```
=====
VISUAL TREE OF COMPUTATION
=====
      2^10
      |
      (2*2)^5  <-- n was 10 (even)
      |
      4 * (4*4)^2 <-- n was 5 (odd), pull one '4' out
      |
      4 * (16*16)^1 <-- n was 2 (even)
      |
      4 * 256 * (256*256)^0 <-- n was 1 (odd), pull '256' out
      |
      4 * 256 * 1 = 1024
```

Handling Negative Exponents If $n < 0$, we transform the problem: $x = 1 / x^n = x^{-n}$ Now we can solve it as a positive exponent problem.

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(\log n)$

In every step of the algorithm, we divide 'n' by 2.
This is exactly how a binary search or a balanced binary tree works.

If $n = 1024$, it takes 10 steps ($2^{10} = 1024$).
If $n = 1,000,000$, it takes ~20 steps.

DERIVATION DIAGRAM:

```
n = 1000 --> step 1
n = 500  --> step 2
n = 250  --> step 3
...
n = 1     --> step 10
Total steps = log base 2 of n.
```

Result: $O(\log n)$

SPACE COMPLEXITY: $O(\log n)$ or $O(1)$

1. RECURSIVE APPROACH:

Each division by 2 adds a new frame to the 'Call Stack'.
Space = $O(\log n)$.

2. ITERATIVE APPROACH (Optimized):

We only use a few variables (current_product, x, n).
No matter how large n is, the memory used is the same.

Result: $O(1)$ for Iterative

4. Solution Code

A top-tier engineer will usually provide the **Iterative** version because it is more memory-efficient and avoids stack overflow on systems with limited recursion depth.

Optimized Solution (Iterative)

```
def myPow(x, n):
    # Handle the negative exponent case
    if n < 0:
        x = 1 / x
        n = -n

    result = 1
    current_product = x

    # We iterate while there is still a power to calculate
    while n > 0:
        # If n is odd, multiply the result by the current_product
        # (This corresponds to pulling one 'x' out in the odd logic)
        if n % 2 == 1:
```

```

        result = result * current_product

        # Square the base for the next step
        current_product = current_product * current_product

        # Integer division of n by 2
        # (Using bitwise shift n >> 1 is also an L5/L6 touch)
        n = n // 2

    return result

/**
 * @param {number} x
 * @param {number} n
 * @return {number}
 */
var myPow = function(x, n) {
    // Handle negative n
    // In JS, we use BigInt logic or simply convert to positive
    // let longN = n;
    if (n < 0) {
        x = 1 / x;
        n = -n;
    }

    let result = 1;
    let currentProduct = x;

    while (n > 0) {
        // If n is odd (check last bit)
        if (n % 2 === 1) {
            result = result * currentProduct;
        }

        // Square the current base
        currentProduct = currentProduct * currentProduct;

        // Move to the next power of 2 by halving n
        // Math.floor is necessary in JS as n/2 produces floats
        n = Math.floor(n / 2);
    }

    return result;
};

```

Note 1: Terminology and Techniques

- **Binary Exponentiation:** Also known as “square-and-multiply.” It uses the binary representation of the exponent to determine when to multiply the accumulated base into the result.
 - **Tail Recursion / Iteration:** The shift from a recursive “Divide and Conquer” to an iterative loop is a hallmark of an engineer who understands stack management and performance optimization.
-

Note 2: Real-World / Interview Variations

1. Modular Exponentiation (Google / Bloomberg) * The Problem: Calculate $(x^n) \% m$. * **The L5 Solution:** You cannot calculate x^n first and then use $\% m$ because x^n will be so large it will overflow any computer memory. Instead, you apply the modulo at **every single multiplication step** inside the while loop: `result = (result * current_product) \% m`.

2. Matrix Exponentiation (Meta / Bloomberg) * The Problem: Calculate the N-th Fibonacci number in $O(\log n)$ time. * **The L5 Solution:** You can represent the Fibonacci sequence as a matrix multiplication. To find the N-th number, you raise a specific 2x2 matrix to the power n. You use the exact same `myPow` logic, but instead of multiplying numbers, you perform matrix multiplication.

3. Distributed Power Calculation (Google - Systems Design) * The Problem: You need to calculate a massive exponentiation where x is a huge matrix that doesn't fit on one machine. * **The L5 Solution:** You would discuss how to parallelize the squaring steps. However, since squaring is sequential (**Step 2** depends on **Step 1**), you would focus on distributing the actual matrix-multiplication task across a cluster (using a framework like MapReduce or a distributed linear algebra library) while the control logic follows the binary exponentiation flow.